

Database Management Systems

CPS 3740

Lecture 15

Computer Science, Kean University

Dr. Ching-yu (Austin) Huang

Instructor Paolien Wang

Outline

- Continue last lecture
- Create view
- Updatable vs un-updatable view
- Team research
- Homework status

Views

View

Dynamic result of one or more relational operations operating on base relations to produce another relation.

- Virtual relation that does not necessarily actually exist in the database but is produced upon request, at time of request.**

Views

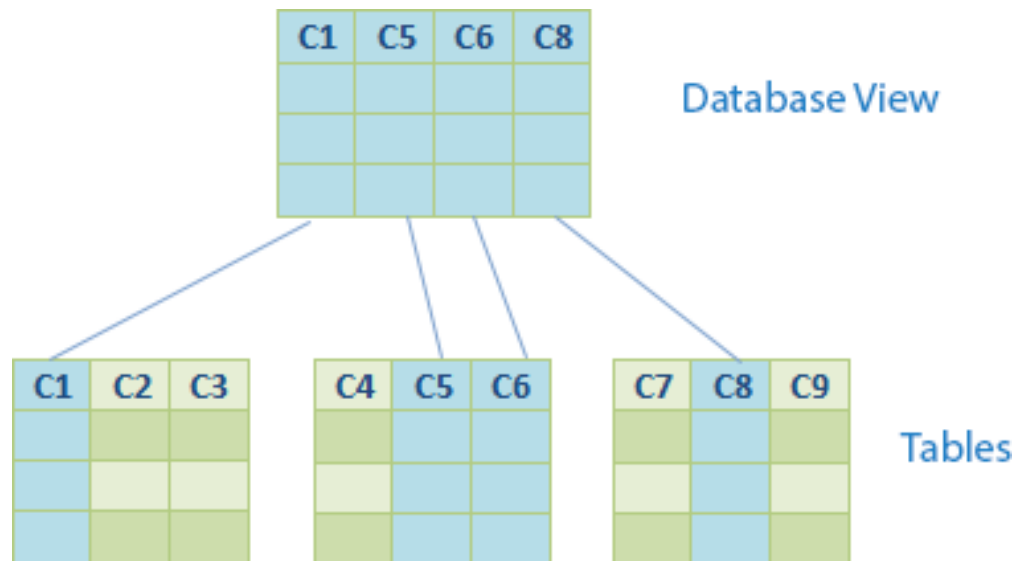
- Contents of a view are defined as a query on one or more base relations.
- With view resolution, any operations on view are automatically translated into operations on relations from which it is derived.
- With view materialization, the view is stored as a temporary table, which is maintained as the underlying base tables are updated.

Views (Virtual Tables) in SQL

- A view in SQL:
 - Is a single table **derived** from other tables
 - Considered to be a **virtual** table
- Views as virtual tables:
 - Generally, a **table** has a set of definitions and physically stores data
 - A **view** has a set of definitions, which is built **on top of table(s) or other view(s)**, but it **does not physically store the data**
- Need to have the **create view** privilege

Views

- Base Relation
 - Named relation corresponding to an entity in conceptual schema, whose tuples are physically stored in database.
- View
 - Dynamic result of one or more relational operations operating on base relations to produce another relation.



Views

- A virtual relation that does not necessarily actually exist in the database but is **produced upon request, at time of request**.
- Contents of a view are defined as **a query on one or more base relations**.
- Views are dynamic, meaning that **changes** made to base relations that affect view attributes are **immediately** reflected in the view.

Purpose of Views

- Provides powerful and flexible security mechanism by hiding parts of database from certain users.
- Permits users to access data in a **customized way**, so that **same data** can be seen by **different users** in **different ways**, at same time.
 - security, especially for internal users
- Can simplify complex operations on base relations.
 - performance, especially for big data

SQL - CREATE VIEW Syntax

```
CREATE VIEW ViewName [ (newColumnName [,...]) ]  
    AS subselect  
    [WITH [CASCADED | LOCAL] CHECK OPTION]
```

- Can assign a name to each column in view.
- If list of column names is specified, it must have **same number** of items as number of columns produced by *subselect*.
- If omitted, each column takes name of corresponding column in *subselect*.
- List must be specified if there is any ambiguity in a column name.

SQL - CREATE VIEW Syntax cont'd

- The *subselect* is known as the defining query.
- WITH CHECK OPTION ensures that if a row fails to satisfy WHERE clause of defining query, it is not added to underlying base table.
- CASCADED causes the checks for underlying views to be evaluated as well. When neither keyword is given, the default is CASCADED. More discussions later.
- Need SELECT privilege on all tables referenced in subselect and USAGE privilege on any domains used in referenced columns.

Example 7.3 - Create Horizontal View

- Create view so that manager at branch B003 can only see details for staff who work in his or her office.

```
CREATE VIEW Manager3Staff
AS      SELECT *
        FROM Staff
        WHERE branchNo = 'B003';
```

staffNo	fName	lName	position	sex	DOB	salary	branchNo
SG37	Ann	Beech	Assistant	F	10-Nov-60	12000.00	B003
SG14	David	Ford	Supervisor	M	24-Mar-58	18000.00	B003
SG5	Susan	Brand	Manager	F	3-Jun-40	24000.00	B003

Example 7.4 - Create Vertical View

- Create view of staff details at branch B003 excluding salaries.

CREATE VIEW Staff3

AS SELECT staffNo, fName, lName, position, sex

FROM Staff

WHERE branchNo = 'B003';

staffNo	fName	lName	position	sex
SG37	Ann	Beech	Assistant	F
SG14	David	Ford	Supervisor	M
SG5	Susan	Brand	Manager	F

Example 7.5 - Grouped and Joined Views

- Create view of staff who manage properties for rent, including branch number they work at, staff number, and number of properties they manage.

```
CREATE VIEW View_xxxx (branchNo, staffNo, cnt)  
AS SELECT s.branchNo, s.staffNo, COUNT(*)  
  FROM Staff s, PropertyForRent p  
  WHERE s.staffNo = p.staffNo  
  GROUP BY s.branchNo, s.staffNo;
```

Results - Grouped and Joined Views

```
select * from PropertyForRent ;
```

propertyNo	street	city	postcode	type	rooms	rent	ownerNo	staffNo	branchNo
PA14	16 Holhead	Aberdeen	AB7 5SU	House	6	650.00	C046	SA9	B007
PG16	5 Novar Drive	Glasgow	G12 9AX	Flat	4	450.00	C093	SG14	B003
PG21	18 Dale Road	Glasgow	G12	House	5	600.00	C087	SG37	B003
PG36	2 Manor Road	Glasgow	G32 4QX	Flat	3	375.00	C093	SG37	B003
PG4	6 Lawrence Street	Glasgow	G11 9QX	Flat	3	350.00	C040	NULL	B003
PL94	6 Argyll Street	London	NW2	Flat	4	400.00	C087	SL41	B005

```
select * from Staff ;
```

staffNo	fName	lName	position	sex	DOB	salary	branchNo
SA9	Mary	Howe	Assistant	F	1970-02-19	9000.00	B007
SG14	David	Ford	Supervisor	M	1958-03-24	18000.00	B003
SG37	Ann	Beech	Assistant	F	1960-11-10	12000.00	B003
SG5	Susan	Brand	Manager	F	1940-06-03	24000.00	B003
SL21	John	White	Manager	M	1945-10-01	30000.00	B005
SL41	Julie	Lee	Assistant	F	1965-06-13	9000.00	B005

branchNo	staffNo	cnt
B003	SG14	1
B003	SG37	2
B005	SL41	1
B007	SA9	1

CREATE VIEW vTest_xxxx (branchNo, staffNo, cnt)
AS SELECT s.branchNo, s.staffNo, COUNT(*)
FROM dreamhome.Staff s,
dreamhome.PropertyForRent p
WHERE s.staffNo = p.staffNo
GROUP BY s.branchNo, s.staffNo;

Specification of Views in SQL

- The records of a view are dynamically generated by the select statement every time
 - Views are always up-to-date
 - Responsibility of the DBMS and not the user
- Can assign a name to each column in a view
- If a list of column names is specified, it must have the same number of items as a number of columns produced by a *subquery*.
- If omitted, each column takes the name of the corresponding column in a *subquery*.
- **DROP VIEW** command
 - Dispose of a view

SQL - DROP VIEW

DROP VIEW ViewName [RESTRICT | CASCADE]

DROP VIEW [IF EXISTS] *view_name*

[, *view_name*] ... [RESTRICT | CASCADE]

- Causes definition of view to be deleted from database.
- For example:

DROP VIEW Manager3Staff;

SQL - DROP VIEW

- **With CASCADE, all related dependent objects are deleted; i.e. any views defined on view being dropped.**
- **With RESTRICT (default), if any other objects depend for their existence on continued existence of view being dropped, command is rejected.**

Class exercise - create table

- xxxx is your Kean email ID. Use classDB (eg. `TECH3740_2023S`)
- Create table `Test_xxxx` that has 2 columns (id, name)
`Create table Test_xxxx (id int, name varchar(20), gender varchar(2) );`
- Get insert/update/delete permissions from the class website

- Add two records

`Insert into Test_xxxx values (1, 'Austin', 'M'),(2,'Mary', 'F');`

- Show records

`Select * from Test_xxxx;`

Class exercise - create view

- **CREATE VIEW vTest_xxxx (id,name, gender)**
AS SELECT id, name, gender
FROM Test_xxxx **WHERE** gender='M';

- Check View

- **Select * from vTest_xxxx;**

- Insert records into table

Insert into Test_xxxx values (3,'Grant','M'),(4,'Sam','F');

Check View again.

Select * from vTest_xxxx; -- what happen?

it should display only **gender='M'**

Class exercise - update view

- Insert records into View

Insert into vView_xxxx values (5,'CCC', 'M'),(6,'AAA', 'F');

- Check Table

Select * from Test_xxxx; -- what happen? 6 records

- Update View will also update table
- Check View again

Select * from vView_xxxx; -- what happen? 3 records

- Check View again. it should still display only **gender='M'**

Class exercise - drop view

- Drop table

Drop **table** Test_xxxx;

- Check View

Select * from vTest_xxxx; -- what happen? 3 records

- If the table is gone, View will reference to invalid table
- Drop View

Drop **View** vTest_xxxx;

Access Control Using View

- How to allow people to access certain rows?
- How to allow people to access certain columns?
- Give certain privileges to people on a view

Can we change/update the views

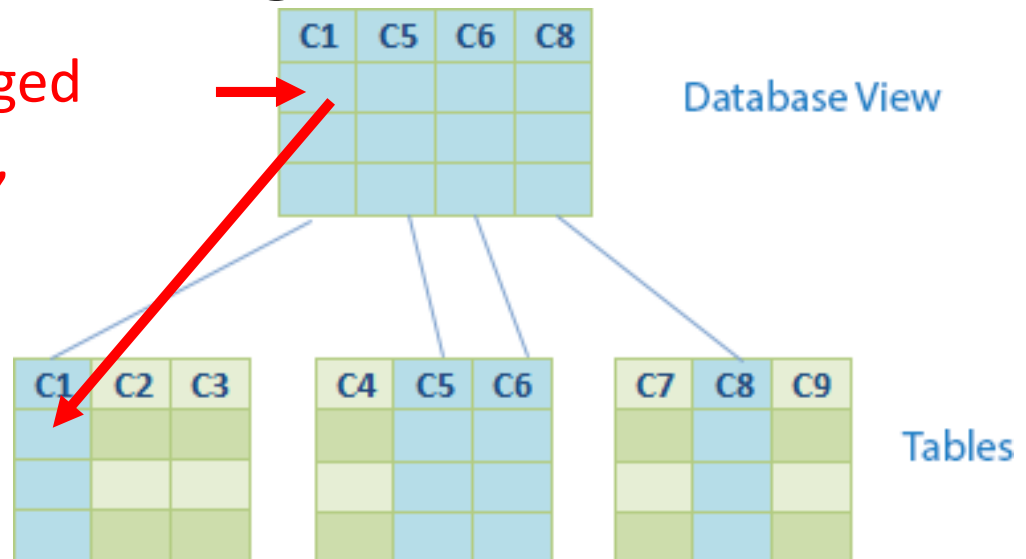
- Since we don't know the underlying tables, what will happen if we try to insert/update/delete the view data?
 - It is depended How the view was created...
- If we can inserting/updating/deleting data on a view, that view is called **updatable view**.

Updating Views

- All updates to a base relation should be immediately reflected in all views that reference that base relation.
- If a view is updatable, underlying base relation should reflect change.

If the value is changed
from 2 to 1 at View,

the value in original table
will be automatically
changed from 2 to 1.



Updating Views

- There are restrictions on types of modifications that can be made through views:
 - Updates are **allowed** if query involves a **single** base relation and contains a candidate key of base relation.
- Updates are **not allowed**:
 - involving **multiple** base relations.
 - involving **aggregation** or **grouping** operations.
 - if the records are **NOT one-to-one** mapped between the view and original tables. (distinct)

Updating Views

- Classes of views are defined as:
 - theoretically not updateable;
 - theoretically updateable;
 - partially updateable.
- What will happen if we update the following views?

Products		
ID	Name	Price
1	iPad	100
2	iPhone	100
3	iTouch	200

P_View1	
ID	Name
1	iPad
2	iPhone
3	iTouch

P_View2	
ID	Price
1	100
2	100
3	200

P_View3
Price
100
200

P_View4
Price_sum
300

P_View5		
ID	Name	Price
3	iTouch	200

Updatable View

- **Updatable view: view can be directly updated**
 - Use UPDATE, INSERT, or DELETE to update the view
 - When view is updated, underlying database also changes
- For a view to be updatable, the DBMS must be able to trace any row or column back to its row or column in the source table.
- **Check how a view was created:**
mysql> show create view vTest_xxxx;

Updateable View

- In order to create an updateable view, the SELECT statement that defines the view has to follow several following rules:
 - The SELECT statement must only refer to **one database table**.
 - The SELECT statement must **not use a GROUP BY or HAVING** clause.
 - The SELECT statement must **not use DISTINCT** in the column list of the SELECT clause.
 - The SELECT statement must **not refer to read-only views**.
 - The SELECT statement must **not contain any expressions** (aggregates, functions, computed columns)

Un-updatable Views

- Views defined using **distinct**, **group by**, and **aggregate functions** are **not updateable**
- Views defined on multiple tables using joins are generally not updateable
- `WITH CHECK OPTION`: must be added to the definition of a view if the view is to be updated (**MySQL does not support CHECK**)
 - To allow check for updatability and to plan for an execution strategy

Example of Create View

id	name	type	price
123451	Park's Great Hits	Music	19.99
123452	Silly Puddy	Toy	3.99
123453	Playstation	Toy	89.95
123454	Men's T-Shirt	Clothing	32.5
123455	Blouse	Clothing	34.97
123456	Electronica 2002	Music	3.99
123457	Country Tunes	Music	21.55
123458	Watermelon	Food	8.73

1. create view **v_items** as
(select **name**, **price** from items);

name	price
Park's Great Hits	19.99
Silly Puddy	3.99
Playstation	89.95
Men's T-Shirt	32.5
Blouse	34.97
Electronica 2002	3.99
Country Tunes	21.55
Watermelon	8.73

2. create view **v_prices** as
(select **distinct price** from items);

price
19.99
3.99
89.95
32.5
34.97
21.55
8.73

Advantages of Views

- **Data independence**
- **Concurrency**
- **Improved security**
- **Reduced complexity**
- **Convenience**
- **Customization**
- **Data integrity**

Disadvantages of Views

- Update restriction
- Structure restriction
 - Can not have subquery in the FROM clause
 - The following view can not be created (check day6's slide for this subquery)

Create view vTest_demo as

SELECT staffNo, fName, lName, position,
s.salary-avg1.savg **as** SalDiff

FROM dreamhome.Staff s,

(**SELECT AVG**(salary) **as** savg **FROM** dreamhome.Staff) **as** avg1

WHERE s.salary > avg1.savg ;

- Performance

Home Exercises – create views 1

- Use MySQL Workbench or putty (Windows), terminal (Mac) to connect to imc.kean.edu
- Set CPS3740_2020S as your default database
`mysql> Use CPS3740_2020S;`
- Assume xxxx is your DB user ID
- Copy dreamhome.Staff table to your own table
`mysql> Create table Staff_xxxx as select * from dreamhome.Staff;`
- Copy dreamhome.Branch table to your own table
`mysql> Create table Branch_xxxx as select * from dreamhome.Branch;`
- Get insert/update/delete privileges on these two tables from the class website. (option 6.3)

Home Exercises – create views 2

- Use your own Staff and Branch tables.
- Create a view vTest_xxxx
 - Show the staffno, fname, lname, branchno for the female staff who has salary > 10000;
`mysql> select * from vTest_xxxx;`
 - Can we see DOB, position columns?
→ **security** (or privacy), cannot see the underlying table data
- Create a view vTest1_xxxx
 - Show the staffno, fname, lname, city for the female
`mysql> select * from vTest1_xxxx;`
 - Do we know how many tables and columns are involved?
→ security + **reduce the complexity** (no join in select the view)

Home Exercises – create views 3

- Create a view vTest2_xxxx
 - Show the branchno, number_of_staff, max_salary, min_salary, avg_salary for each branch and the output is from high to low based on the avg_salary
- ```
mysql> select * from vTest2_xxxx;
```
- Can we see the salary for each staff?
  - Do we know how many tables and columns are involved?
  - Do we know how the results were produced?
    - The results will be **consistent** for different departments
  - The summary view is very common used in data warehouse
  - Everyone gets the same results from the formula (SQL statement)

# Home Exercises – create views 4

- Add one female staff into your own Staff table, branchno='B002'
  - Insert into Staff\_xxxx values ('SG20','Sarah','Brown','Assistant','F',Date '1999-10-10',12000,'B002');
- Check your view vTest\_xxxx
  - `mysql> select * from vTest_xxxx;`
  - Do you see the new staff?
    - View always gets updated (the newest data)
- Check your view vTest1\_xxxx
  - The result includes the new staff and her working city
    - The join is automatically done by the database
- Check your view vTest2\_xxxx
  - The result shows the new summary
    - The join and calculation are automatically done by the DB

# Home Exercises – create views 5

- Insert data to vTest\_xxxx ( updatable view)

Insert vTest\_xxxx values ('SG60','Linda','Lin','B002');

- Check your table Staff\_xxxx

```
mysql> select * from Staff_xxxx;
```

- Do you see the new staff?

- Check your view vTest\_xxxx

```
mysql> select * from vTest_xxxx;
```

- Do you see the new staff? No. why?

- Check the view creation.

- show create view vTest\_demo;

=> view is created with

where ((`Staff\_xxxx`.`sex` = 'F') and (`Staff\_xxxx`.`salary` > 10000))

- Reason: Linda has no salary data.

# Home Exercises – create views 6

- Insert data to vTest1\_xxxx ( non-updatable view)

```
Insert vTest1_xxxx (staffno, fname, lname, city)
values ('SG21','Nancy','Brown','London');
```

<error message>

Can not modify more than one base table through a join  
view 'TECH3740\_2021S1.vTest1\_xxxx'

# Homework

- Review the requirements
- We will cover stored routines soon.

# Reading Assignment

- Understand View
- Understand Updatable view vs. non-updatable view
- Do the all exercises on the slides and textbook
- SQL exercise paper



# Conclusions

- Work on Project
- Do more exercises on Views
- Work on the team research ASAP